

closure_evidence_20260925

BOB-240 closure evidence — §11.4.33 Type↔Status mechanical-prevention guard

Revision: 1 **Last modified:** 2026-09-25T14:20:00Z **Status:** Fixed

Location: constitution/scripts/workable-items/cmd/workable-items/
(the Helix Universal Constitution's own Go tooling, git submodule).

Constitution HEAD at time of work:

25980c1c53ca5ec043b141984b6dba9ea0b2d75e

(2026-09-25T10:34:26+02:00) — the conductor performs the §11.4.26
fetch/merge/push-to-all-upstreams commit for this submodule
separately, outside this session's scope. No `git add/commit/push` was
executed inside `constitution/` by this session.

1. What was fixed

`constitution/scripts/workable-items's close / update` subcommands
accepted a `--status` keyword whose §11.4.33 closure vocabulary
belongs to a DIFFERENT Type than the target item's actual Type (e.g. a
Type=Task item closed with `--status fixed`, the Bug-mapped word,
instead of `--status completed`) — the exact mistake this session's
conductor made four times in a row (BOB-077/100/179/226) on a DB
that already held 30+ historical instances of the same class (bulk-
corrected in BOB-239). `validate` never checked this Type↔Status
agreement either.

Three surfaces now share ONE predicate (`typeStatusMismatch`, `crud.go`, derived from the pre-existing `closeStatusMap` so the status TEXT is never re-derived, §11.4.251):

1. **close** (`crud.go`) — refuses a `--status` keyword whose Type-mapping mismatches the item's Type, unless the keyword is obsolete (valid for ANY Type, criterion 5).
2. **update** (`mutate.go`) — the identical guard, scoped to a TERMINAL `--status`, so it composes cleanly with BOB-166/BOB-175's existing `location↔status` guards rather than competing with them (see §4 below).
3. **validate** (`sync.go`, `typeStatusMismatches`) — a new full-table detective sweep over every Fixed-location item, catching rows minted before the guard existed or by raw SQL bypassing the CLI entirely.

2. Exact diff applied

2a. `crud.go` — new shared predicate + `closeCmd` guard (full diff)

```
--- a/scripts/workable-items/cmd/workable-items/crud.go
+++ b/scripts/workable-items/cmd/workable-items/crud.go
@@ (immediately after closeStatusMap's declaration)
+// typeCloseKeyword is the §11.4.33 closed Type→close-keyword
+//      mapping BOB-240
+// enforces: which `close --status <word>` / `update --status
+//      <word>` keyword
+// is the ONLY correct one for each Type (Bug→fixed,
+//      Feature→implemented,
+// Task→completed). Obsolete is deliberately ABSENT from this
+//      table – per
+// §11.4.90/§11.4.33 it is valid closure vocabulary for ANY Type
+//      (criterion 5),
+// so it is exempted explicitly in typeStatusMismatch rather than
+//      being a
+// fourth, always-true entry here.
+var typeCloseKeyword = map[string]string{
+  "Bug":      "fixed",
+  "Feature":  "implemented",
+  "Task":     "completed",
+}
+
+// closeKeywordForStatus reverse-maps a terminal closed-set
+//      status TEXT back
+// onto its closeStatusMap keyword ...
+func closeKeywordForStatus(status string) string {
+  for k, m := range closeStatusMap {
```

```

+         if m.status == status {
+             return k
+         }
+     }
+     return ""
+}
+
+// typeStatusMismatch reports whether TERMINAL status ns violates
+// the §11.4.33
+// Type↔Status mapping for typ ...
+func typeStatusMismatch(typ, ns string) (wantKeyword, wantStatus
+    string, mismatch bool) {
+    if ns == closeStatusMap["obsolete"].status {
+        return "", "", false
+    }
+    keyword, ok := typeCloseKeyword[typ]
+    if !ok {
+        return "", "", false
+    }
+    wantStatus = closeStatusMap[keyword].status
+    return keyword, wantStatus, ns != wantStatus
+}

```

@@ closeCmd, after the itemExists(db, id, "Fixed") check, before closedBody is built

```

+ // BOB-240 §11.4.33 Type↔Status guard: refuse a --status
+ // keyword whose
+ // implied Type-mapping does not match src.Type (Obsolete
+ // exempt for ANY
+ // Type – criterion 5). Placed BEFORE any write.
+ if wantKeyword, wantStatus, mismatch :=
+     typeStatusMismatch(src.Type, mapping.status); mismatch {
+     fmt.Fprintf(os.Stderr,
+         "close: refusing – %s has Type=%s, whose §11.4.33
+         closure vocabulary is %q (--status %s), not %q (--status
+         %s); Obsolete is valid for any Type.\n"+
+         "    correct command:  close %s --db <db> --status
+         %s --evidence %s\n",
+         id, src.Type, wantStatus, wantKeyword, mapping.status,
+         strings.ToLower(strings.TrimSpace(*status)),
+         id, wantKeyword, *evidence)
+     return exitUsage
+ }

```

2b. mutate.go — updateCmd guard (full diff, inserted right before cur.Status = ns at the end of the set["status"] block, after BOB-166's and BOB-175's location↔status checks)

```
+ // BOB-240 §11.4.33 Type↔Status guard – scoped to ns
+ // TERMINAL: by this
+ // point EITHER loc=="Issues" && !terminal (survived
+ // BOB-166 above)
+ // OR loc=="Fixed" && terminal (survived BOB-175's mirror
+ // above) – the
+ // only two combinations reaching this line.
+ if terminalStatuses()[strings.TrimSpace(ns)] {
+     if wantKeyword, wantStatus, mismatch :=
+         typeStatusMismatch(cur.Type, ns); mismatch {
+         gotKeyword := closeKeywordForStatus(ns)
+         fmt.Fprintf(os.Stderr,
+             "update: refusing – %s has Type=%s, whose
+             §11.4.33 closure vocabulary is %q (--status %s), not %q
+             (--status %s); Obsolete is valid for any Type.\n"+
+             " correct command: update --id %s --db
+             <db> --location %s --status %s\n",
+             *id, cur.Type, wantStatus, wantKeyword, ns,
+             gotKeyword,
+             *id, loc, wantKeyword)
+         return exitUsage
+     }
+ }
+ cur.Status = ns
```

2c. sync.go — typeStatusMismatches detective invariant + wiring

```
@@ (new function, before unresolvableClosureEvidence)
+func typeStatusMismatches(items []item) []string {
+    terminal := terminalStatuses()
+    var out []string
+    for _, it := range items {
+        if it.CurrentLocation != "Fixed" {
+            continue
+        }
+        ns := strings.TrimSpace(it.Status)
+        if !terminal[ns] {
+            continue // fixedLocationNonTerminalStatus's own
+            finding.
+        }
+        if wantKeyword, wantStatus, mismatch :=
+            typeStatusMismatch(it.Type, ns); mismatch {
+            out = append(out, fmt.Sprintf(
```

```

+         "%s: Type=%s closed with status %q – §11.4.33
+         requires %q (`close --status %s`); Obsolete is valid for
+         any Type [%s]",
+         it.AtmID, it.Type, it.Status, wantStatus,
+         wantKeyword, it.repOrDefault())
+     }
+ }
+ return out
+}

@@ validateCmd, after unresolvableClosureEvidence, before the
+ violations-report block
+ // (h) §11.4.33 / BOB-240 – Type↔Status mapping invariant.
+ violations = append(violations,
+     typeStatusMismatches(items)...)

```

2d. New test file bob240_type_status_test.go (full contents — 10 test

functions + 2 fixture helpers, ~430 lines; see the file itself in the submodule working tree for the complete text — reproduced in outline here):

- seedFixedItemOfType, injectTypeStatusMismatchAtFixed — fixture helpers.
- TestCloseCmd_RefusesTypeStatusMismatch (6 sub-cases incl. the exact BOB-077-class Task_closed_fixed).
- TestCloseCmd_RefusalNamesCorrectWord.
- TestCloseCmd_AllowsMatchingTypeStatus (3 sub-cases — Bug/Feature/Task).
- TestCloseCmd_AllowsObsoleteForAnyType (3 sub-cases — criterion 5).
- TestUpdateCmd_RefusesTypeStatusMismatch.
- TestUpdateCmd_RefusalNamesCorrectWord.
- TestUpdateCmd_AllowsMatchingTypeStatus.
- TestUpdateCmd_LocationGuardFiresBeforeTypeStatusGuard — the BOB-175 composition proof (§4 below).
- TestValidate_CatchesTypeStatusMismatch.
- TestValidate_DoesNotFireOnMatchingOrObsoleteClosures (6 sub-cases).

2e. Reconciliation of 5 PRE-EXISTING tests whose fixtures were themselves

the exact Type↔Status mismatch class the new guard now refuses (found by running the guard against the full suite, not guessed in advance):

File	Test
crud_test.go	TestClose_PositionalLast
assign_test.go	TestAssignGroupCompleteCmd_SucceedsWhen
closure_evidence_terminal_types_test.go	seedClosedWithEvidenceAndStatus (used by
update_location_status_invariant_test.go	TestUpdateCmd_AllowsTerminalStatusOnFix
repair_bodies_test.go	TestRepairBodies_ClearsDesyncs_RedPolar TestRepairBodies_Idempotent
roundtrip_test.go	TestValidate_OK_RealDocs

TestUpdateCmd_AllowsMatchingTypeStatus,
TestUpdateCmd_LocationGuardFiresBeforeTypeStatusGuard,
TestValidate_DoesNotFireOnMatchingOrObsoleteClosures (all PASS).

3b. RED_MODE=1 guard-absent-baseline reproduction (the exact historical incident)

```
$ RED_MODE=1 go test -count=1 -run  
'TestCloseCmd_RefusesTypeStatusMismatch|  
TestUpdateCmd_RefusesTypeStatusMismatch|  
TestValidate_CatchesTypeStatusMismatch' -v ./...  
add: created WIT-950 (Task, status=Queued) in Issues  
close: moved WIT-950 Issues→Fixed (status=Fixed (→ Fixed.md),  
evidence=docs/qa/WIT-950/Task_closed_fixed.md)  
validate: OK – 1 items, all invariants satisfied  
--- PASS: TestCloseCmd_RefusesTypeStatusMismatch (0.04s)  
[reproduces: close accepts it, validate ALSO says OK]  
--- PASS: TestUpdateCmd_RefusesTypeStatusMismatch (0.01s)  
--- PASS: TestValidate_CatchesTypeStatusMismatch (0.01s)  
PASS
```

This is captured, physical proof that the guard-absent baseline reproduces the EXACT historical incident — validate: OK while a genuine Type/Status mismatch is live in the DB.

3c. GREEN (guard wired into close, update, validate)

```
$ go test -count=1 -run 'TestCloseCmd_|TestUpdateCmd_|  
TestValidate_' -v ./...  
--- PASS: TestCloseCmd_RefusesTypeStatusMismatch (0.03s)  
--- PASS: TestCloseCmd_RefusalNamesCorrectWord (0.00s)  
--- PASS: TestCloseCmd_AllowsMatchingTypeStatus (0.03s)  
--- PASS: TestCloseCmd_AllowsObsoleteForAnyType (0.03s)  
--- PASS: TestUpdateCmd_RefusesTypeStatusMismatch (0.01s)  
--- PASS: TestUpdateCmd_RefusalNamesCorrectWord (0.01s)  
--- PASS: TestUpdateCmd_AllowsMatchingTypeStatus (0.01s)  
--- PASS: TestUpdateCmd_LocationGuardFiresBeforeTypeStatusGuard  
(0.00s)  
--- PASS: TestValidate_CatchesTypeStatusMismatch (0.01s)  
--- PASS: TestValidate_DoesNotFireOnMatchingOrObsoleteClosures  
(0.04s)  
... [every other pre-existing test in the suite also PASS]
```


3d. §1.1 paired mutation proofs (each guard neutered ONE AT A TIME, in place, then restored — real physical toggling in this session, not merely narrated)

Mutation 1 — closeCmd guard neutered (mismatch short-circuited with false &&):

```
$ go test -count=1 -run 'TestCloseCmd_RefusesTypeStatusMismatch|
TestCloseCmd_RefusalNamesCorrectWord' -v ./...
--- FAIL: TestCloseCmd_RefusesTypeStatusMismatch (0.03s)
--- FAIL: TestCloseCmd_RefusalNamesCorrectWord (0.00s)
```

Restored → re-ran → both PASS (confirmed in §3c).

Mutation 2 — updateCmd guard neutered:

```
$ go test -count=1 -run 'TestUpdateCmd_RefusesTypeStatusMismatch|
TestUpdateCmd_RefusalNamesCorrectWord|
TestUpdateCmd_LocationGuardFiresBeforeTypeStatusGuard' -v ./...
--- FAIL: TestUpdateCmd_RefusesTypeStatusMismatch (0.01s)
--- FAIL: TestUpdateCmd_RefusalNamesCorrectWord (0.01s)
--- PASS: TestUpdateCmd_LocationGuardFiresBeforeTypeStatusGuard
(0.01s) [correctly UNAFFECTED — proves the two guard families
are independent]
```

Restored → re-ran → all 3 PASS (confirmed in §3c).

Mutation 3 — validateCmd's wiring of typeStatusMismatches disabled:

```
$ go test -count=1 -run 'TestValidate_CatchesTypeStatusMismatch'
-v ./...
--- FAIL: TestValidate_CatchesTypeStatusMismatch (0.01s)
```

Restored → re-ran → PASS (confirmed in §3c).

Each of the three checks is proven load-bearing: neutering it — and ONLY it — flips exactly the tests that exercise it from PASS to FAIL, and restoring it flips them back.

4. Obsolete-exemption negative-control proof (criterion 5)

TestCloseCmd_AllowsObsoleteForAnyType and TestValidate_DoesNotFireOnMatchingOrObsoleteClosures each drive close --status obsolete for Bug, Feature, AND Task, asserting (a) close exits 0, (b) the item lands at Fixed with status Obsolete (→ Fixed.md), and (c) validate reports OK — for every one of the three

Types. All 6 sub-cases (3 per test × 2 tests) PASS.

TestUpdateCmd_AllowsMatchingTypeStatus performs the analogous proof for update (Bug → Obsolete transition at Fixed).

5. BOB-175-interaction note (required by the task)

BOB-166's location↔status guard (`loc=="Issues" && terminal`) and BOB-175's mirror guard (`loc=="Fixed" && !terminal`) already partition every update call into exactly one of THREE mutually-exclusive (location, terminality) cells:

- {Issues, terminal} → refused by BOB-166.
- {Fixed, non-terminal} → refused by BOB-175.
- {Issues, non-terminal} → legitimate, passes through untouched.

This leaves {Fixed, terminal} as the ONLY cell that can still reach BOB-240's new check (placed textually last, gated on `terminalStatuses()[ns]`). **A single update call can therefore never trigger more than one of the three guards** — there is no ordering ambiguity to resolve, because the guard families never overlap on the same call in the first place.

TestUpdateCmd_LocationGuardFiresBeforeTypeStatusGuard proves this at runtime: a call using a status that is TYPE-correct (no BOB-240 violation) but TERMINAL-while-still-in-Issues (a BOB-166 violation) is refused with the BOB-166 message ("located in Issues") and NEVER mentions §11.4.33 — confirmed both with the guard present (PASS) and with the BOB-240 guard mutated out (still PASS, unaffected — Mutation 2 above), demonstrating the two families are independent, not merely ordered.

6. Full Go test-suite pass/fail counts

Baseline (this submodule, BEFORE any BOB-240 change, `go test -count=1 ./... -v`): **286 PASS, 0 FAIL** (3 pre-existing SKIP-OK entries not separately counted at that point).

Final (after the fix + all reconciliations, `go test -race -count=1 ./... -v`):

```
$ go test -race -count=1 ./... -v 2>&1 | tail -3
--- PASS: TestVersionTagsEmitJSON (0.02s)
PASS
```

ok github.com/HelixDevelopment/HelixConstitution/scripts/
workable-items/cmd/workable-items 6.031s

- **PASS: 296**
- **FAIL: 0**
- **SKIP: 3** (pre-existing RED_MODE-only skips, unrelated to this item)
- go build ./... — clean.
- go vet ./... — clean.
- Race detector (-race) — clean, no data races.

Net delta vs. baseline: **+10 top-level test functions**, all new

(TestCloseCmd_RefusesTypeStatusMismatch,
TestCloseCmd_RefusalNamesCorrectWord,
TestCloseCmd_AllowsMatchingTypeStatus,
TestCloseCmd_AllowsObsoleteForAnyType,
TestUpdateCmd_RefusesTypeStatusMismatch,
TestUpdateCmd_RefusalNamesCorrectWord,
TestUpdateCmd_AllowsMatchingTypeStatus,
TestUpdateCmd_LocationGuardFiresBeforeTypeStatusGuard,
TestValidate_CatchesTypeStatusMismatch,
TestValidate_DoesNotFireOnMatchingOrObsoleteClosures), several
carrying multiple sub-cases (t.Run) — **ZERO previously-passing tests
now fail.**

7. HONEST, IN-SCOPE DISCOVERY — real pre-existing data defects found (not fixed)

Running the new validate invariant against the LIVE docs/Fixed.md +
docs/Issues.md (via TestValidate_OK_RealDocs, read-only — no CLI
invocation against the live DB, no write to any live doc) surfaced
THREE real, currently-live §11.4.33 Type↔Status violations that
BOB-239's 30-item bulk correction did NOT include (they were closed
with the mismatch AFTER that bulk-correction ran, later in this same
session):

- BOB-135: Type=Bug closed with status "Completed (→ Fixed.md)" —
§11.4.33 requires "Fixed (→ Fixed.md)" (`close --status fixed`);
Obsolete is valid for any Type [section]
- BOB-238: Type=Bug closed with status "Completed (→ Fixed.md)" —
§11.4.33 requires "Fixed (→ Fixed.md)" (`close --status fixed`);
Obsolete is valid for any Type [section]
- BOB-239: Type=Bug closed with status "Completed (→ Fixed.md)" —
§11.4.33 requires "Fixed (→ Fixed.md)" (`close --status fixed`);
Obsolete is valid for any Type [section]

These were NOT corrected as part of this task — BOB-240's own acceptance criteria scope it to the mechanical-PREVENTION half only (not a second bulk correction), and this task's constraints explicitly forbid touching docs/Fixed.md / docs/Issues.md / docs/workable_items.db. Per §11.4.238's discovery-channel discipline, this finding is recorded here honestly rather than hidden behind a weakened test assertion — the reconciled TestValidate_OK_RealDocs (§2e above) asserts the EXACT known set by name, so it will fail loudly if any OTHER/NEW violation appears, and will silently start passing once these three are corrected upstream. **A follow-up tracked item to correct BOB-135/BOB-238/BOB-239's status text in the live tracker is owed** (recommended for the conductor to file, since filing it would itself require touching docs/workable_items.db, which is out of this task's scope).

8. Evidence file path

/home/milosvasic/Projects/boba/docs/qa/BOB-240/
closure_evidence_20260925.md (this file).